James
Posted on Nov 5, 2024 • Edited on Nov 14, 2024

12

Streamlit Part 6: Mastering Layouts

#python #programming #streamlit #webdev

Streamlit 101 (9 Part Series)

1 Streamlit Part 1: Write and Text Elements

2 Streamlit Part 2: Mastering Data Elements for Interactive ...

... **5 more parts...**

8 Streamlit Part 8: Status Elements

9 Streamlit Part 10: Page Navigation Simplified

Mastering Layouts in Streamlit: A Step-by-Step Guide

Streamlit, a widely-used framework for building interactive Python applications, particularly for data visualization, dashboards, and machine learning demos, stands out not only for its user-friendly nature but also for its ability to create visually appealing and intuitive layouts. In this blog post, we'll guide you through a Python example that demonstrates how to effectively utilize layout elements such as columns, containers, placeholders, and more in Streamlit.

Let's break down the layout techniques you can use to make your apps cleaner and more interactive.

Setting the Stage: Page Configuration

Before jumping into the layout elements, we configure the page using `st.set_page_config()`. This method allows you to customize the page title, icon, layout, and sidebar behavior right when the app loads.

```
st.set_page_config(
    page_title="Streamlit Layouts Tutorial",
    page_icon=":art:",
    layout="wide",
    initial_sidebar_state="collapsed",
)
```

Here, we give the page a title, set the layout to "wide" (which makes use of the full browser width), and collapse the sidebar initially for a cleaner look.

1. Structuring with Columns

One of the most powerful layout tools in Streamlit is columns. They allow you to display content side-by-side, giving a more organized and visually appealing look to your app.

```
st.header("Columns")
st.write("Using `st.columns()` to create columns.")

# Create two columns
col1, col2 = st.columns(2)

col1.write("This is column 1")
if col1.button("Button in Column 1"):
    col1.write("Button 1 pressed")

col2.write("This is column 2")
```

```
if col2.button("Button in Column 2"):
    col2.write("Button 2 pressed")
```

In this snippet, we create two columns and place buttons in each. The columns are split evenly, and any interactions within one column don't affect the other.

Why Columns?

Columns are great for displaying related information side by side, such as data summaries, charts, or interactive controls.

2. Grouping with Containers

Next up is the container element. Containers in Streamlit allow you to group multiple elements together, making it easier to manage complex layouts.

```
st.header("Container")
st.write("Using `st.container()` to group elements together.")

with st.container():
    st.write("This is inside a container")
    st.button("Button inside container")

    # Nested container
    with st.container():
        st.write("This is a nested container")
        st.button("Button inside nested container")
```

In this example, the `st.container()` method wraps multiple elements (text and a button) together. You can even nest containers inside one another to create hierarchical layouts.

Why Containers?

Containers help maintain a clean and grouped structure, especially when dealing with multiple sections of content that belong together logically.

3. Dynamically Updating with Placeholders

A powerful feature of Streamlit is its ability to update content dynamically. This is done using `st.empty()`, which serves as a placeholder that you can update later.

```
st.header("Empty")
st.write("Using `st.empty()` as a placeholder for updating conten

placeholder = st.empty()

# Update the placeholder content dynamically
for i in range(5):
    placeholder.write(f"Updating... {i}")
    time.sleep(1)

placeholder.write("Done!")
```

In this example, we use a `for` loop to update the placeholder with a new value every second. Once the loop is done, we replace the placeholder content with "Done!"

Why Use Placeholders?

Placeholders are ideal for situations where you need to update parts of your app dynamically without rerunning the entire app, such as live data feeds or progress updates.

4. Hiding and Showing with Expanders

Expandable sections are perfect for hiding advanced settings or additional information that you don't want to clutter the main layout.

```
st.header("Expander")
st.write("Using `st.expander()` to hide/show content.")

with st.expander("Click to expand"):
    st.write("This is inside the expander")
    st.button("Button inside expander")
    ecol1, ecol2 = st.columns(2)
    with ecol1:
        st.write("This is column 1")
    with ecol2:
        st.write("This is column 2")
```

Here, we wrap some content and a button inside an expander, which users can click to reveal or hide the content.

Why Expanders?

Expanders help keep your interface clean by hiding less important or advanced options while still making them easily accessible when needed.

5. Creating Forms

Streamlit forms allow you to group input widgets together and wait for the user to submit them all at once, rather than reacting to each input individually.

```
st.header("Form")
st.write("Using `st.form()` to group input widgets with a submit

with st.form("my_form"):
    st.write("Inside the form")
    name = st.text_input("Enter your name")
    age = st.number_input("Enter your age", min_value=0, max_valu
    submitted = st.form_submit_button("Submit")
    if submitted:
        st.write(f"Hello {name}, you are {age} years old.")
```

In this example, we use a form to collect a user's name and age, and only once they click the submit button does Streamlit process the input.

Why Forms?

Forms ensure that input actions are grouped and submitted as a batch, which is ideal for cases like user registration or data filtering.

6. Adding a Sidebar

Sidebars are useful for navigation, app settings, or extra options that don't clutter the main interface.

```
st.header("Sidebar")
st.write("Using `st.sidebar` to add content to the sidebar.")

with st.sidebar:
    st.write("This is in the sidebar")
    st.button("Sidebar Button")
```

This code adds a button to the sidebar, which collapses by default but can be expanded by the user.

Why Use Sidebars?

Sidebars are perfect for secondary content like navigation links, filters, or app settings that are always accessible but don't need to take up space in the main layout.

7. Navigating with Tabs

Tabs are a great way to organize content within a single section, allowing users to switch between different views without leaving the page.

```
st.header("Tabs")
st.write("Using `st.tabs()` to create tabbed sections.")

tab1, tab2, tab3 = st.tabs(["Cat", "Dog", "Owl"])

with tab1:
    st.write("This is the cat tab")
    st.image("<https://static.streamlit.io/examples/cat.jpg>", wi

with tab2:
    st.write("This is the dog tab")
    st.image("<https://static.streamlit.io/examples/dog.jpg>", wi

with tab3:
    st.write("This is the owl tab")
    st.image("<https://static.streamlit.io/examples/owl.jpg>", wi
```

In this example, we use three tabs to display different content related to animals. Each tab is independent and contains its own content.

Why Tabs?

Tabs are ideal for organizing related content into sections, like different data views or categories of information, without requiring separate pages.

Conclusion

Mastering Streamlit's layout elements empowers you to create clean, interactive, and user-friendly applications. By skillfully using columns, containers, placeholders, expanders, forms, sidebars, and tabs, you can effectively organize your content and enhance the overall user experience. These tools allow you to craft intuitive interfaces that guide users through your application seamlessly.

🔗 Get the Code: GitHub - jamesbmour/blog_tutorials
🔗 Related Streamlit Tutorials:JustCodeIt
☕ Support my work: Buy Me a Coffee

Streamlit 101 (9 Part Series)

1 Streamlit Part 1: Write and Text Elements

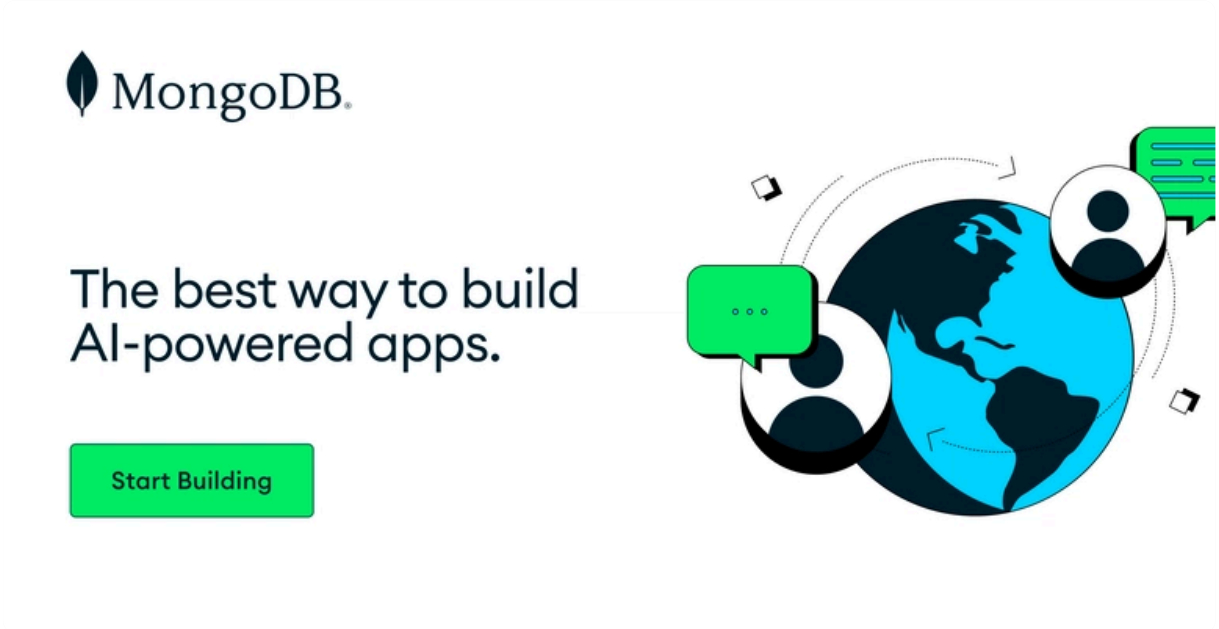
2 Streamlit Part 2: Mastering Data Elements for Interactive ...

... **5 more parts...**

8 Streamlit Part 8: Status Elements

9 Streamlit Part 10: Page Navigation Simplified

MongoDB PROMOTED



Build gen AI apps that run anywhere with MongoDB Atlas

MongoDB Atlas bundles vector search and a flexible document model so developers can build, scale, and run gen AI apps without juggling multiple databases. From LLM to semantic search, Atlas streamlines AI architecture. Start free today.

Start Free

Top comments (0)

Code of Conduct • Report abuse

Sonar PROMOTED

State of Code Developer Survey report

Did you know 96% of developers don't fully trust that AI-generated code is functionally correct, yet only 48% always check it before committing? Check out Sonar's new report on the real-world impact of AI on development teams.

Read the results

James

If you would like to support my writing or buy me a beer: https://buymeacoffee.com/bmours

LOCATION
Denver

EDUCATION

Ohio State - ISE , Georgia institute of technology - Masters in CS

WORK

Freelance Consulting

JOINED

Jan 23, 2024

More from James

Building a Human-in-the-Loop AI App with LangGraph and Ollama

#python #ai #llm #langgraph

Interrupts and Commands in LangGraph: Building Human-in-the-Loop Workflows

#programming #python #ai #langgraph

Building a News and Stock Assistant with LangGraph and Chainlit

#programming #ai #tutorial #python

MongoDB PROMOTED

MongoDB.

MongoDB Atlas runs apps anywhere

Start Building